

Security & Spring Security Complete Interview Guide

Simple, Clear, Visual — Deep Explanations, Real Implementations,
Real-Time Scenarios, Common Issues, Fixes & Interview Q&A

Table of Contents

1. What is Application Security? (Basics)
2. Authentication vs Authorization
3. Common Attacks You Must Know
4. Introduction to Spring Security
5. Spring Security Internal Architecture (Deep Dive)
6. Spring Security Filter Chain (Visual Flow)
7. Setting Up a Full Project (Step by Step)
8. In-Memory Authentication (Simple Example)
9. Database Authentication with Custom UserDetailsService
10. Form Login & Session-based Auth (Deep Dive)
11. Password Encoding & Storage (Deep Dive)
12. JWT (Token-Based) Authentication — Full Implementation
13. Refresh Token Implementation
14. OAuth2 / Login with Google (Full Flow)
15. Role-Based Access Control (RBAC) — Deep Dive
16. Method Security Deep Dive (@PreAuthorize, @PostAuthorize, @Secured)
17. Custom Filters & Exception Handling
18. CORS, CSRF & Security Headers (Deep Dive)
19. Multi-Factor Authentication (MFA)
20. Securing Microservices & API Gateway
21. Testing Spring Security
22. Actuator & Production Security
23. Real-World Case Study 1: Banking Application
24. Real-World Case Study 2: Healthcare Application
25. Real-World Case Study 3: E-commerce Application
26. Common Issues & Fixes (Extended)
27. Security Best Practices Checklist
28. Glossary of Terms
29. Interview Questions & Answers (30 Q&A)
30. Quick Revision Summary

1. What is Application Security? (Basics)

Security means making sure that **only the right people** can access **only the right data/actions**, and that data is protected from being stolen, changed, or misused. Every application — a banking app, a shopping site, a hospital system — needs to answer three simple questions before doing anything sensitive:

- **Who are you?** → Authentication
- **What are you allowed to do?** → Authorization
- **Is the data safe while traveling/stored?** → Encryption

Real-time example: At a bank, showing your ID card at the door is **Authentication** (proving who you are). Being allowed only into the "customer area" and not the "vault" is **Authorization** (what you can access). The locked bulletproof glass around the cash counter is like **Encryption** (protecting data/assets even if someone gets close to it).

In software, these three ideas map directly onto Spring Security's job:

- Authentication → Login forms, JWT tokens, OAuth2, Basic Auth
- Authorization → Roles, permissions, `@PreAuthorize`, URL rules
- Encryption → HTTPS/TLS in transit, password hashing (BCrypt) at rest

2. Authentication vs Authorization

Authentication	Authorization
Verifies WHO you are (login with username/password, OTP, token)	Verifies WHAT you can do (roles/permissions)
Happens first	Happens after authentication
Failure → 401 Unauthorized	Failure → 403 Forbidden
Example: Logging into Gmail	Example: Only Admin can delete a user
Managed by <code>AuthenticationManager</code>	Managed by <code>AccessDecisionManager</code> / <code>AuthorizationManager</code>

Simple memory trick: Authentication = "Login". Authorization = "Permission". If a security guard checks your badge at the entrance, that's authentication. If he then tells you "you can only enter Floor 2, not Floor 5", that's authorization.

3. Common Attacks You Must Know

Attack	Simple Meaning	How Spring Security Helps
SQL Injection	Attacker inserts SQL code into an input field to manipulate the database.	Use JPA/Prepared Statements (parameter binding), never string-concatenate SQL.
XSS (Cross-Site Scripting)	Attacker injects malicious JavaScript that runs in another user's browser.	Escape/encode output, Content-Security-Policy header.
CSRF (Cross-Site Request Forgery)	Tricks a logged-in user's browser into submitting an unwanted request.	CSRF tokens for session-based apps (built into Spring Security).
Session Fixation	Attacker forces a victim to use a known session ID, then hijacks it after login.	Spring Security regenerates session ID automatically after login.
Clickjacking	Tricks users into clicking something different from what they see, using invisible iframes.	<code>X-Frame-Options</code> header (enabled by default).
Brute Force	Repeatedly trying passwords until one works.	Account lockout, rate limiting, CAPTCHA (custom implementation).
Man-in-the-Middle	Attacker intercepts communication between client and server.	Enforce HTTPS/TLS everywhere, HSTS header.

Real-time example: In 2017, Equifax suffered a massive data breach because a known web framework vulnerability was left unpatched. Lesson: keeping dependencies updated (Spring Boot, Spring Security versions) is itself a critical security practice.

4. Introduction to Spring Security

Spring Security is a framework that plugs into a Spring Boot app and handles authentication, authorization, and protection against common attacks (CSRF, session fixation, clickjacking) — mostly through configuration, not manual code.

Maven dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

As soon as you add this dependency, Spring Boot auto-secures ALL endpoints with a login form and a generated password (shown in console) — this is the "secure by default" idea. This is intentional: Spring Security wants you to explicitly open up endpoints rather than accidentally leave something unprotected.

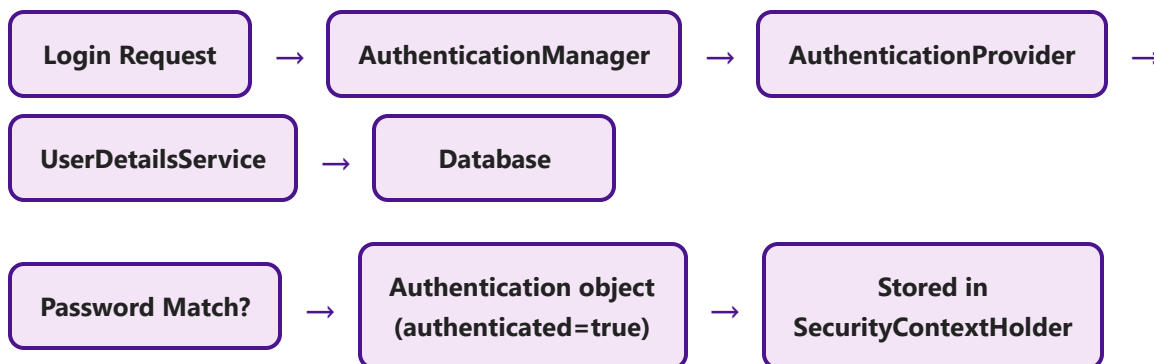
Why is Spring Security so widely used?

- Battle-tested — used in thousands of production systems for over a decade.
- Highly configurable — supports session-based, token-based, OAuth2, SAML, LDAP.
- Integrates natively with the rest of the Spring ecosystem (Spring MVC, Spring Data, Spring Boot Actuator).
- Handles the "boring but critical" plumbing so developers focus on business logic.

5. Spring Security Internal Architecture (Deep Dive)

Understanding what happens internally makes debugging much easier. Here are the key building blocks:

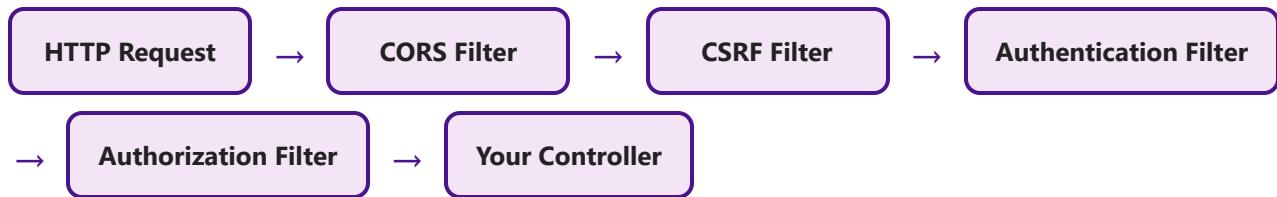
Component	Role
<code>SecurityContextHolder</code>	Stores the currently authenticated user for the current thread — accessible anywhere via <code>SecurityContextHolder.getContext().getAuthentication()</code> .
<code>Authentication</code>	An object representing the current user — holds principal (user), credentials, and granted authorities.
<code>AuthenticationManager</code>	The main interface that attempts to authenticate — delegates to one or more <code>AuthenticationProvider</code> s.
<code>AuthenticationProvider</code>	Does the actual check (e.g., <code>DaoAuthenticationProvider</code> checks username/password against a database).
<code>UserDetailsService</code>	Loads user-specific data (username, password, roles) — you implement this to connect to your own DB.
<code>UserDetails</code>	Represents the loaded user (implements username, password, authorities, account-status flags).
<code>GrantedAuthority</code>	Represents a single permission/role, e.g. <code>ROLE_ADMIN</code> .
<code>SecurityFilterChain</code>	The ordered list of servlet filters applied to every incoming request.



Simple summary: Think of `AuthenticationManager` as the "manager" who doesn't personally check your ID but delegates it to a "specialist" (`AuthenticationProvider`), who asks the "HR records" (`UserDetailsService`) to look up your file, then compares passwords.

6. Spring Security Filter Chain (Visual Flow)

Every request passes through a chain of filters before reaching your controller.



If any filter rejects the request (bad token, no login, no permission), it stops there and returns 401 (Unauthorized) or 403 (Forbidden) — the controller code never even runs. This "fail fast" behavior protects your business logic from ever seeing unauthenticated traffic.

Some important built-in filters (in typical order)

Filter	Purpose
<code>SecurityContextPersistenceFilter</code>	Loads/saves the <code>SecurityContext</code> for each request.
<code>CorsFilter</code>	Handles cross-origin requests.
<code>CsrfFilter</code>	Validates CSRF tokens for state-changing requests.
<code>UsernamePasswordAuthenticationFilter</code>	Handles form login POST submissions.
<code>BasicAuthenticationFilter</code>	Handles HTTP Basic Auth headers.
<code>BearerTokenAuthenticationFilter</code> / Custom JWT Filter	Extracts and validates a bearer token.
<code>ExceptionHandlerFilter</code>	Catches security exceptions and converts them to 401/403 responses.
<code>FilterSecurityInterceptor</code> / <code>AuthorizationFilter</code>	Makes the final authorization decision (allow/deny).

7. Setting Up a Full Project (Step by Step)

Here is a minimal but complete Spring Boot + Spring Security project structure:

```
src/main/java/com/example/demo
├─ DemoApplication.java
├─ config/SecurityConfig.java
├─ controller/AuthController.java
├─ controller/ProductController.java
├─ model/User.java
├─ repository/UserRepository.java
├─ security/CustomUserDetailsService.java
├─ security/JwtUtil.java
└─ security/JwtAuthFilter.java
```

Step 1 — pom.xml dependencies

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
  </dependency>
</dependencies>
```

Step 2 — User entity

```
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String username;
    private String password;
    private String role; // e.g. "ADMIN", "USER"
```

```
// getters and setters  
}
```

Step 3 — Repository

```
public interface UserRepository extends JpaRepository<User, Long> {  
    Optional<User> findByUsername(String username);  
}
```

With this structure in place, we now build authentication step by step, starting with the simplest form (in-memory) before moving to real database + JWT.

8. In-Memory Authentication (Simple Example)

This is the easiest way to try Spring Security for learning/demo purposes — user data lives only in memory, not a real database.

```
@Configuration  
public class SecurityConfig {  
  
    @Bean  
    public UserDetailsService userDetailsService() {  
        UserDetails user = User.withUsername("john")  
            .password("{noop}password123") // {noop} = no encoding, for demo only  
            .roles("USER")  
            .build();  
        UserDetails admin = User.withUsername("admin")  
            .password("{noop}admin123")  
            .roles("ADMIN")  
            .build();  
        return new InMemoryUserDetailsManager(user, admin);  
    }  
}
```

WARNING {noop} stores the password as plain text — only ever use this for quick local demos, never in production.

Real-time example: Teams often use in-memory auth to quickly spin up a demo/POC before the real user database and login screen are ready — it lets frontend developers start integrating immediately.

9. Database Authentication with Custom UserDetailsService

In real applications, users live in a database. You implement `UserDetailsService` to tell Spring Security how to load a user by username.

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPassword()) // already BCrypt-hashed in DB
            .roles(user.getRole())
            .build();
    }
}
```

```
@Configuration
public class SecurityConfig {

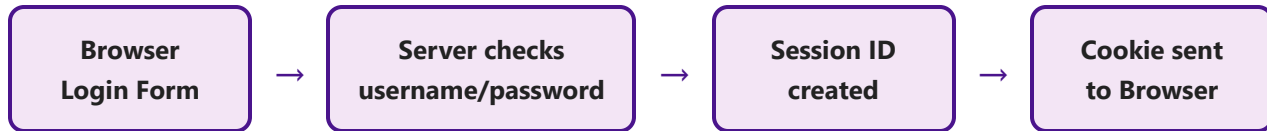
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider(CustomUserDetailsService uds,
PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(uds);
        provider.setPasswordEncoder(encoder);
        return provider;
    }
}
```

Real-time example: A company intranet portal stores 5,000 employees in a MySQL "users" table with hashed passwords and a "department" role. When an employee logs in, Spring Security calls `loadUserByUsername` ,

fetches their record, and compares the hashed password — no plaintext passwords ever touch memory or logs.

10. Form Login & Session-based Auth (Deep Dive)



On every next request, the browser automatically sends the session cookie, and the server looks up the session to know who you are. This is the classic "stateful" web app approach (used in traditional server-rendered apps like Thymeleaf/JSP).

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/login", "/css/**").permitAll()
            .anyRequest().authenticated())
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/dashboard", true)
            .permitAll())
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout"));
    return http.build();
}
```

Session lifecycle

1. User submits login form → Spring Security authenticates.
2. A new `HttpSession` is created (with a new random session ID to prevent session fixation).
3. Session ID is sent to the browser as a cookie (`JSESSIONID`).
4. Every subsequent request auto-includes this cookie; server maps it back to the authenticated user.
5. Session expires after inactivity (configurable) or on explicit logout.

Real-time example: Logging into an internal HR portal — after login, a `JSESSIONID` cookie is stored in your browser; every page you click after that reuses this cookie so you don't need to log in again, until you close the browser or the session times out (e.g., after 30 minutes of inactivity).

When to use session-based auth: Best for traditional server-rendered web apps where frontend and backend are the same application. Not ideal for mobile apps or multiple microservices, where JWT is usually preferred.

11. Password Encoding & Storage (Deep Dive)

Never store plain-text passwords. Spring Security provides `BCryptPasswordEncoder` which hashes the password with a random "salt" so even two identical passwords produce different hashes.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(); // strength factor default = 10
}

// saving user
user.setPassword(passwordEncoder.encode(rawPassword));
userRepository.save(user);

// Spring Security automatically calls encoder.matches(rawPassword, storedHash) during login
```

Why not MD5/SHA-1?

MD5 and SHA-1 are fast — great for checksums, terrible for passwords, because attackers can try billions of guesses per second using GPUs. BCrypt is deliberately slow and includes a random salt, making brute-force attacks impractical.

Algorithm	Good for Passwords?	Why
MD5	No	Fast, no salt, broken/collidable
SHA-256	No (alone)	Still too fast for password hashing without extra work
BCrypt	Yes	Slow by design, automatic salting, industry standard
Argon2	Yes (modern choice)	Winner of Password Hashing Competition, resistant to GPU/ASIC attacks

ISSUE

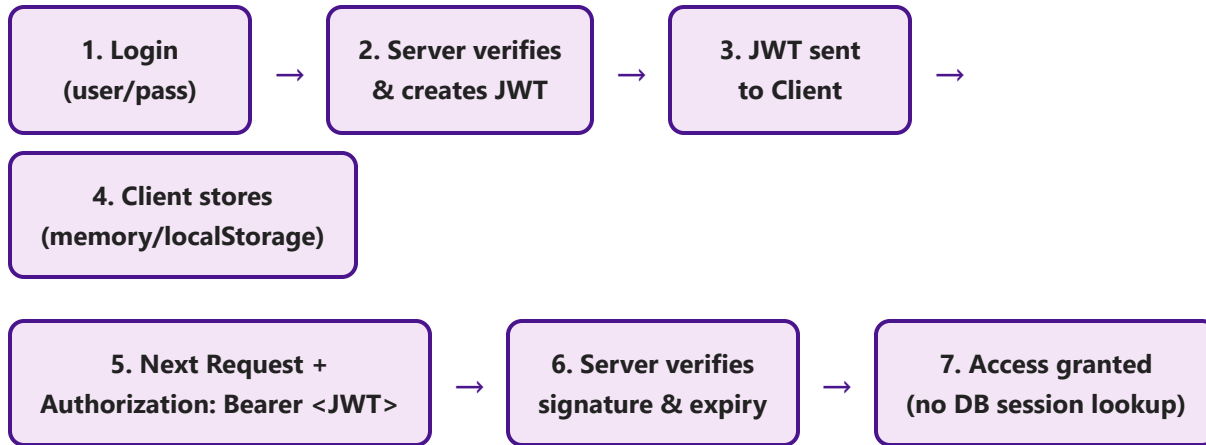
 Storing passwords as plain text or using weak hashing (MD5/SHA1 without salt).

FIX

 Always use BCrypt/Argon2 with Spring's `PasswordEncoder` abstraction — never write your own hashing logic.

12. JWT (Token-Based) Authentication — Full Implementation

JWT (JSON Web Token) is used mostly for REST APIs and Single Page Apps (React, Angular) where the server should NOT store session state (stateless).



JWT structure (3 parts, separated by dots)

HEADER.PAYLOAD.SIGNATURE

eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJqb2huIiwicm9sZSI6IkFETU10In0 . SflKxwRJSMeKKF2QT4f...

- **Header** — algorithm used (e.g., HS256)
- **Payload** — user info/claims (username, role, expiry) — NOT encrypted, just encoded (never put secrets here)
- **Signature** — proves the token wasn't tampered with (signed using a secret key)

Step 1 — JwtUtil (generate & validate tokens)

```

@Component
public class JwtUtil {
    private final String SECRET = "your-256-bit-secret-key-should-be-in-env-var";
    private final long EXPIRATION = 1000 * 60 * 60; // 1 hour

    public String generateToken(String username, String role) {
        return Jwts.builder()
            .setSubject(username)
            .claim("role", role)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION))
            .signWith(Keys.hmacShaKeyFor(SECRET.getBytes()), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {

```

```

        return getClaims(token).getSubject();
    }

    public boolean isTokenValid(String token) {
        try {
            return !getClaims(token).getExpiration().before(new Date());
        } catch (JwtException e) {
            return false;
        }
    }

    private Claims getClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(Keys.hmacShaKeyFor(SECRET.getBytes()))
            .build().parseClaimsJws(token).getBody();
    }
}

```

Step 2 — JwtAuthFilter (custom filter to check token on every request)

```

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired private JwtUtil jwtUtil;
    @Autowired private CustomUserDetailsService userService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
chain)
        throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            String username = jwtUtil.extractUsername(token);

            if (username != null && jwtUtil.isTokenValid(token)
                && SecurityContextHolder.getContext().getAuthentication() == null) {

                UserDetails userDetails = userService.loadUserByUsername(username);
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
    }
}

```

```

    }
    chain.doFilter(req, res);
}
}

```

Step 3 — Register the filter & make app stateless

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http, JwtAuthFilter jwtAuthFilter) throws
Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}

```

Step 4 — Login endpoint that issues the JWT

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired private AuthenticationManager authManager;
    @Autowired private JwtUtil jwtUtil;
    @Autowired private UserRepository userRepository;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(req.getUsername(), req.getPassword()));

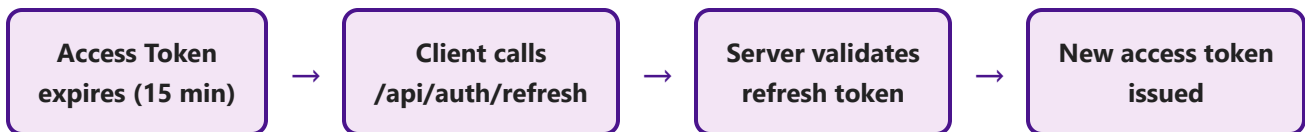
        User user = userRepository.findByUsername(req.getUsername()).get();
        String token = jwtUtil.generateToken(user.getUsername(), user.getRole());
        return ResponseEntity.ok(new AuthResponse(token));
    }
}

```

Real-time example: In a React app calling `/api/login`, receives a JWT, stores it, and sends it as `Authorization: Bearer <token>` header on every API call to the Spring Boot backend. The backend never stores session — fully stateless, great for scaling across multiple servers behind a load balancer.

13. Refresh Token Implementation

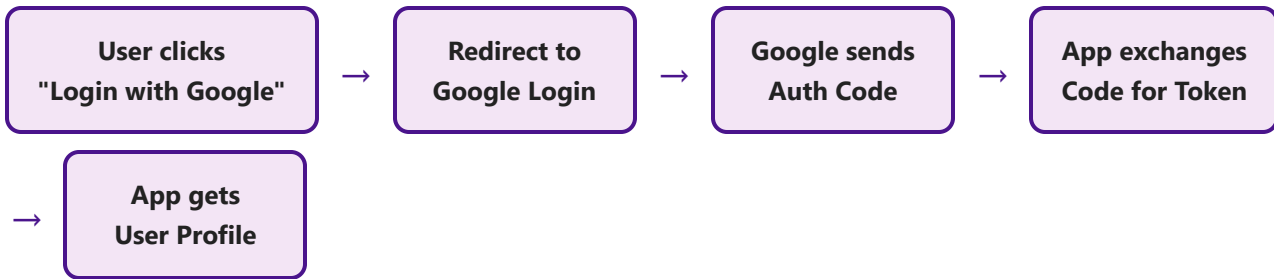
Access tokens are kept short-lived (e.g., 15 minutes) for security. A longer-lived **refresh token** is used to get a new access token without forcing the user to log in again.



```
@PostMapping("/refresh")
public ResponseEntity<> refresh(@RequestBody RefreshRequest req) {
    if (!jwtUtil.isRefreshTokenValid(req.getRefreshToken())) {
        return ResponseEntity.status(401).body("Invalid refresh token");
    }
    String username = jwtUtil.extractUsername(req.getRefreshToken());
    User user = userRepository.findByUsername(username).get();
    String newAccessToken = jwtUtil.generateToken(user.getUsername(), user.getRole());
    return ResponseEntity.ok(new AuthResponse(newAccessToken));
}
```

Store refresh tokens server-side (DB/Redis) so they can be revoked on logout or if compromised — unlike access tokens, refresh tokens should be revocable.

14. OAuth2 / "Login with Google" (Full Flow)



Dependency

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>

```

application.yml

```

spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: YOUR_GOOGLE_CLIENT_ID
            client-secret: YOUR_GOOGLE_CLIENT_SECRET
            scope: profile, email

```

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .anyRequest().authenticated())
        .oauth2Login(oauth -> oauth.defaultSuccessUrl("/home", true));
    return http.build();
}

```

Real-time example: Many SaaS products offer "Sign in with Google/GitHub" so users don't need to create yet another password. Spring Boot handles the entire OAuth2 handshake with just these config lines — no manual redirect/token-exchange code needed.

15. Role-Based Access Control (RBAC) — Deep Dive

```

@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/users")
public List<User> getAllUsers() { ... }

@PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
@PostMapping("/reports")
public Report generateReport() { ... }

```

Java Config example

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .requestMatchers("/api/public/**").permitAll()
        .anyRequest().authenticated()
    )
    .csrf(csrf -> csrf.disable())
    .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}

```

RBAC vs Fine-Grained Permissions

Simple RBAC works when roles map cleanly to permissions (Admin can do everything, User can only view). For complex apps, you may need **permission-based** checks (e.g., "can_edit_invoice") instead of coarse roles, often combined with a custom `PermissionEvaluator`.

Real-time example: In a hospital management system, a "Doctor" role can view/edit patient records, but a "Receptionist" role can only view names and appointment slots — enforced purely through role checks, same API, different access.

16. Method Security Deep Dive

Enable method security with a single annotation on your config class:

```
@Configuration
@EnableMethodSecurity // enables @PreAuthorize, @PostAuthorize, @Secured
public class MethodSecurityConfig { }
```

Annotation	When it Runs	Example
@PreAuthorize	Before method executes	@PreAuthorize("hasRole('ADMIN')")
@PostAuthorize	After method executes (can check the return value)	@PostAuthorize("returnObject.owner == authentication.name")
@Secured	Before method executes, simpler syntax, no SpEL	@Secured("ROLE_ADMIN")
@PreFilter / @PostFilter	Filters a collection argument/return value	@PostFilter("filterObject.owner == authentication.name")

```
@PostAuthorize("returnObject.ownerUsername == authentication.name")
public Order getOrder(Long id) {
    return orderRepository.findById(id).orElseThrow();
}
// Ensures a user can only view their OWN order, even if they guess another order's ID
```

Real-time example: A "view invoice" API checks the URL path only for role (Authenticated user), but uses @PostAuthorize to make sure the invoice actually belongs to the logged-in customer — preventing "Insecure Direct Object Reference" (IDOR) attacks, where User A tries to view User B's invoice by changing the ID in the URL.

17. Custom Filters & Exception Handling

When authentication fails or access is denied, Spring Security lets you customize the exact JSON response instead of the default HTML error page — critical for REST APIs.

```
@Component
public class CustomAuthEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest req, HttpServletResponse res,
        AuthenticationException ex)
        throws IOException {
        res.setContentType("application/json");
        res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        res.getWriter().write("{\"error\": \"Unauthorized - please login\"}");
    }
}

@Component
public class CustomAccessDeniedHandler implements AccessDeniedHandler {
    @Override
    public void handle(HttpServletRequest req, HttpServletResponse res, AccessDeniedException
        ex)
        throws IOException {
        res.setContentType("application/json");
        res.setStatus(HttpServletResponse.SC_FORBIDDEN);
        res.getWriter().write("{\"error\": \"Forbidden - insufficient permissions\"}");
    }
}
```

```
http.exceptionHandling(ex -> ex
    .authenticationEntryPoint(customAuthEntryPoint)
    .accessDeniedHandler(customAccessDeniedHandler));
```

18. CORS, CSRF & Security Headers (Deep Dive)

Concept	Simple Meaning
CORS	Browser rule that blocks a webpage on domain A from calling an API on domain B unless the server explicitly allows it.
CSRF	Attack where a malicious site tricks your browser into submitting a request to another site where you're logged in (using your cookie).
XSS	Attacker injects malicious JavaScript into a page that other users view.

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("https://myfrontend.com"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

CSRF protection matters mainly for cookie/session-based apps. For stateless JWT REST APIs, CSRF is usually disabled since there's no cookie to hijack.

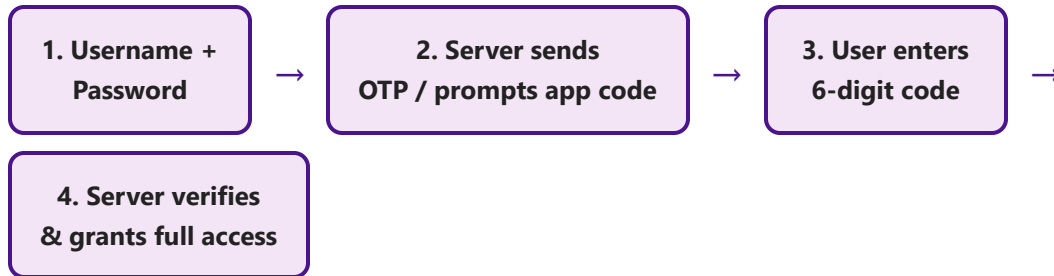
Important security headers

Header	Purpose
Strict-Transport-Security	Forces browser to always use HTTPS (HSTS).
X-Frame-Options	Prevents your site from being embedded in another site's iframe (anti-clickjacking).
X-Content-Type-Options: nosniff	Stops the browser from guessing content types, preventing certain XSS tricks.
Content-Security-Policy	Restricts which scripts/styles/resources a page is allowed to load.

```
http.headers(headers -> headers
    .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'"))
    .frameOptions(frame -> frame.deny()));
```

19. Multi-Factor Authentication (MFA)

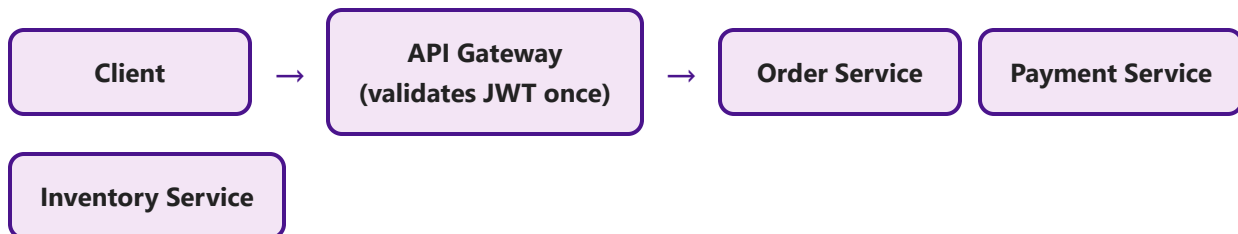
MFA adds a second proof of identity beyond just a password — usually a one-time code (OTP) sent via SMS/email or generated by an authenticator app (TOTP).



Real-time example: Logging into online banking asks for your password first, then sends an OTP to your registered mobile number — the transfer money option is blocked until both steps succeed. This is implemented as a two-stage authentication flow, often with a short-lived "partially authenticated" token between steps.

20. Securing Microservices & API Gateway

In a microservices system, you typically don't want every single service to re-implement login. Instead:



Each downstream microservice acts as a "Resource Server" — it just validates the JWT signature (using a shared secret or public key) and trusts the claims inside (user id, roles) without re-checking the database on every call.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.mycompany.com
```

Real-time example: Netflix-style architecture: the API Gateway authenticates once using a central Identity Provider (like Keycloak/Okta), then forwards a signed JWT to each internal microservice, which independently verifies the token without an extra network call back to the login service.

21. Testing Spring Security

```
@WebMvcTest(ProductController.class)
@Import(SecurityConfig.class)
class ProductControllerTest {

    @Autowired MockMvc mockMvc;

    @Test
    @WithMockUser(roles = "ADMIN")
    void adminCanAccessDashboard() throws Exception {
        mockMvc.perform(get("/admin/dashboard"))
            .andExpect(status().isOk());
    }

    @Test
    void unauthenticatedUserIsRejected() throws Exception {
        mockMvc.perform(get("/admin/dashboard"))
            .andExpect(status().isUnauthorized());
    }
}
```

`@WithMockUser` lets you simulate a logged-in user with specific roles without needing a real login flow in your test — very useful for quickly testing authorization rules.

22. Actuator & Production Security

Spring Boot Actuator exposes endpoints like `/actuator/health` and `/actuator/env` — the latter can leak secrets if left open publicly.

```
management:
  endpoints:
    web:
      exposure:
        include: health,info    # don't expose everything by default
```

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/actuator/health").permitAll()
    .requestMatchers("/actuator/**").hasRole("ADMIN"));
```

ISSUE Leaving `/actuator/env` or `/actuator/heapdump` publicly accessible in production.

FIX Only expose `health` and `info` publicly; secure everything else behind authentication.

23. Real-World Case Study 1: Banking Application

Requirements: Extremely high security, MFA, strict session timeouts, full audit logging, encrypted data at rest.

- Login uses password + OTP (MFA) before any account access.
- All money-transfer APIs require re-authentication (step-up auth) even within an active session.
- Session timeout after 5 minutes of inactivity.
- Every action (view balance, transfer) is logged with user ID, IP, and timestamp for audit.
- All traffic forced over HTTPS with HSTS; database columns for account numbers encrypted.

Implementation detail: Sensitive endpoints use a short-lived "step-up" JWT claim (`"mfa_verified": true`) that expires in 2 minutes — money transfer APIs check this claim explicitly with `@PreAuthorize("#jwt.claims['mfa_verified'] == true")` -style logic.

24. Real-World Case Study 2: Healthcare Application

Requirements: HIPAA-style compliance — patient data must only be seen by authorized medical staff, with fine-grained access.

- Roles: Doctor, Nurse, Receptionist, Patient — each with different visible fields.
- Doctors can only view patients assigned to them (not the whole hospital) — enforced with `@PostAuthorize` ownership checks.
- All access to medical records is logged (who viewed what, when) for compliance audits.
- Patient portal uses JWT (stateless), internal staff dashboard uses session-based login (higher control over session revocation).

Implementation detail: A `PermissionEvaluator` checks `"doctor.assignedPatients.contains(patientId)"` before allowing record access — going beyond simple roles into per-record ownership rules.

25. Real-World Case Study 3: E-commerce Application

Requirements: Public browsing, but secure checkout, admin product management, and integration with a React SPA and a mobile app.

- Product listing/search APIs are public (`permitAll()`).
- Cart & checkout APIs require login (JWT-based, works for both web and mobile).
- Admin product management restricted to `ROLE_ADMIN` .
- Payment APIs re-validate price/stock server-side — never trust the frontend's total.
- Rate limiting on login endpoint to prevent credential-stuffing attacks.

Implementation detail: Login endpoint uses a simple in-memory or Redis-based counter — after 5 failed attempts from the same IP within 10 minutes, further attempts are blocked with 429 Too Many Requests, mitigating brute-force login attempts.

26. Common Issues & Fixes (Extended)

ISSUE **401 Unauthorized** even with correct credentials.

FIX Check password encoder mismatch (raw vs encoded), or wrong `UserDetailsService` bean; verify token isn't expired.

ISSUE **403 Forbidden** even after successful login.

FIX Missing role/authority on user, or CSRF token missing for state-changing (POST/PUT) requests in session-based apps.

ISSUE CORS error: "No 'Access-Control-Allow-Origin' header".

FIX Add a `CorsConfigurationSource` bean and allow your frontend's exact origin (not just `*` if using credentials/cookies).

ISSUE JWT works locally but fails in production (signature invalid).

FIX Secret key differs between environments — store secret in environment variable / vault, keep it identical across instances.

ISSUE Session lost after server restart / load balancer switch.

FIX Use stateless JWT, or externalize sessions (Redis session store) for horizontally scaled apps.

ISSUE Password reset / registration endpoint blocked by security filter.

FIX Add these public endpoints to `permitAll()` explicitly in the security config.

ISSUE Storing JWT in `localStorage` exposes it to XSS attacks.

FIX Prefer `HttpOnly` secure cookies for tokens where possible, sanitize all user input, use `Content-Security-Policy` headers.

ISSUE "Circular bean dependency" error when wiring `PasswordEncoder` into `AuthenticationManager`.

FIX Expose `AuthenticationManager` as a bean via `AuthenticationConfiguration.getAuthenticationManager()` rather than constructing it manually.

ISSUE `@PreAuthorize` annotations are silently ignored.

FIX Forgot to add `@EnableMethodSecurity` on a configuration class.

ISSUE Old Spring Security tutorials use `WebSecurityConfigurerAdapter` which no longer compiles.

FIX `WebSecurityConfigurerAdapter` is removed since Spring Security 5.7+; use a `SecurityFilterChain` bean instead.

ISSUE Users can access another user's data by simply changing the ID in the URL (IDOR).

FIX Add ownership checks with `@PostAuthorize` or explicit service-layer checks — never rely on "hard to guess" IDs alone.

27. Security Best Practices Checklist

- Always use HTTPS/TLS in production — never send credentials or tokens over plain HTTP.
- Hash passwords with BCrypt/Argon2 — never store or log plain-text passwords.
- Keep JWT secrets and DB credentials in environment variables or a secrets vault, never hardcoded.
- Use short-lived access tokens + refresh tokens, not long-lived tokens.
- Validate and re-check everything server-side — never trust frontend calculations.
- Apply the principle of least privilege — give each role only the access it truly needs.
- Log security-relevant events (login, failed login, permission denied) for auditing.
- Keep Spring Boot/Spring Security dependencies updated to patch known vulnerabilities.
- Rate-limit sensitive endpoints (login, password reset) to prevent brute-force attacks.
- Use DTOs to control exactly what data leaves your API — never expose raw entities.

28. Glossary of Terms

Term	Meaning
Principal	The currently logged-in user/identity.
Credentials	Proof of identity (e.g., password, token).
Authority / Role	A permission or group a user belongs to (e.g., ROLE_ADMIN).
Claim	A piece of information embedded inside a JWT (e.g., username, role, expiry).
Access Token	Short-lived token used to access protected resources.
Refresh Token	Longer-lived token used to obtain a new access token.
Resource Server	A service that validates tokens and serves protected resources.
Identity Provider (IdP)	A central service (e.g., Keycloak, Okta, Google) that authenticates users and issues tokens.
Salt	Random data added to a password before hashing to make each hash unique.

29. Interview Questions & Answers (30 Q&A)

Q1. What is Spring Security?

A: A configurable framework that adds authentication and authorization to Spring applications, along with protection against common web attacks like CSRF and session fixation.

Q2. Difference between authentication and authorization?

A: Authentication confirms identity (who you are); authorization decides what actions/data that identity can access.

Q3. What is a SecurityFilterChain?

A: A chain of servlet filters Spring Security registers that intercept every HTTP request to apply authentication/authorization rules before it reaches the controller.

Q4. Why use JWT over session for REST APIs?

A: JWT is stateless — no server-side session storage needed, so it scales easily across multiple servers/load balancers; session-based auth requires sticky sessions or a shared session store.

Q5. What does @PreAuthorize do?

A: It's a method-level security annotation that checks a SpEL expression (like `hasRole('ADMIN')`) before allowing the method to execute.

Q6. How does Spring Security store/verify passwords?

A: Using a `PasswordEncoder` (commonly BCrypt). On login, it re-hashes the entered password and compares hashes — it never decrypts the stored one.

Q7. What is CSRF and when should it be disabled?

A: Cross-Site Request Forgery — an attack abusing browser cookies to perform unwanted actions. It should stay enabled for cookie/session based apps, but is typically disabled for stateless, token-based REST APIs since there's no cookie to forge.

Q8. What is the difference between `hasRole()` and `hasAuthority()`?

A: `hasRole('ADMIN')` automatically expects/prepends "ROLE_" prefix internally; `hasAuthority('ROLE_ADMIN')` requires the exact string. Functionally similar, just different naming convention.

Q9. How do you make an API stateless in Spring Security?

A: Set `sessionCreationPolicy(SessionCreationPolicy.STATELESS)` in the security config, so Spring Security never creates or uses an `HttpSession`.

Q10. What is a custom `UserDetailsService` used for?

A: To tell Spring Security how to load a user (and their roles/passwords) from your own database instead of the

default in-memory user.

Q11. How do you protect against Brute-force login attacks?

A: Account lockout after N failed attempts, CAPTCHA, rate limiting per IP, and delays between attempts.

Q12. What is method-level vs URL-level security?

A: URL-level secures whole endpoints in the filter chain config (e.g., `/admin/**`); method-level uses annotations like `@PreAuthorize` directly on service/controller methods for finer control.

Q13. How does the backend validate a JWT on each request?

A: A custom filter extracts the token from the `Authorization` header, verifies its signature with the secret/public key, checks expiry, then sets the authenticated user into the `SecurityContext`.

Q14. What's the risk of storing JWT in localStorage vs cookie?

A: localStorage is vulnerable to XSS (any injected script can read it); HttpOnly cookies can't be read by JavaScript but are vulnerable to CSRF unless protected with SameSite/CSRF tokens. Choice depends on threat model.

Q15. How do you refresh an expired JWT without forcing re-login?

A: Issue a short-lived access token + a longer-lived refresh token; when the access token expires, the client calls a `/refresh` endpoint with the refresh token to get a new access token.

Q16. What replaced WebSecurityConfigurerAdapter?

A: A `@Bean` of type `SecurityFilterChain` that takes an `HttpSecurity` object — the adapter class was deprecated and removed in Spring Security 5.7+ in favor of component-based configuration.

Q17. What is the SecurityContextHolder?

A: A thread-local storage that holds the current user's `Authentication` object, accessible anywhere in the request-processing thread via `SecurityContextHolder.getContext().getAuthentication()`.

Q18. What is an AuthenticationProvider?

A: A component that performs the actual authentication check for a specific mechanism — e.g. `DaoAuthenticationProvider` checks username/password against a `UserDetailsService` and `PasswordEncoder`.

Q19. How do you prevent IDOR (Insecure Direct Object Reference) attacks?

A: Never rely only on role checks — also verify the requested resource actually belongs to/should be visible to the logged-in user, typically with an ownership check in the service layer or `@PostAuthorize`.

Q20. What is session fixation and how does Spring Security prevent it?

A: An attack where an attacker sets a known session ID for a victim, then hijacks it after the victim logs in. Spring Security prevents this by generating a brand-new session ID immediately after successful authentication.

Q21. What's the difference between @Secured and @PreAuthorize?

A: `@Secured` supports only simple role checks with no expressions; `@PreAuthorize` supports full Spring Expression Language (SpEL), enabling complex conditions like combining roles with method parameters.

Q22. How would you secure Actuator endpoints in production?

A: Restrict exposure to only necessary endpoints (like `health`, `info`) via configuration, and require authentication/ADMIN role for the rest, or expose them on a separate internal-only management port.

Q23. What is a resource server in OAuth2 terms?

A: A service that hosts protected resources/APIs and validates incoming access tokens (often JWTs) issued by a separate Authorization Server, without handling login itself.

Q24. How do you implement "logout" properly with JWT?

A: Since JWTs are stateless and can't be "deleted" server-side by default, you either keep a token blacklist (Redis) checked on each request, use very short expiry with refresh tokens, or simply have the client discard the token (acceptable for low-risk apps).

Q25. What does @EnableMethodSecurity do?

A: It activates method-level security annotations (`@PreAuthorize`, `@PostAuthorize`, `@PreFilter`, `@PostFilter`) across the application context.

Q26. Why should you never expose JPA entities directly as API responses in a secure app?

A: Entities may carry sensitive fields (password hashes, internal flags) or trigger lazy loading issues; DTOs let you explicitly control what's exposed to the client.

Q27. How does rate limiting help security, and where would you add it?

A: It restricts how many requests a client/IP can make in a time window, mitigating brute-force and denial-of-service attempts — typically added at the API Gateway level or via a filter/interceptor on sensitive endpoints like login.

Q28. What's the purpose of the "aud" and "exp" claims in a JWT?

A: "exp" (expiration) defines when the token becomes invalid; "aud" (audience) specifies which service/application the token is intended for, helping prevent token misuse across unrelated services.

Q29. How do you test a secured REST endpoint without a real login?

A: Use `@WithMockUser(roles = "ADMIN")` in a Spring MVC test to simulate an authenticated user with given roles, avoiding the need to perform a real login flow.

Q30. What is "defense in depth" in application security?

A: Applying multiple independent layers of protection (network firewall, HTTPS, authentication, authorization, input validation, encryption) so that if one layer fails, others still protect the system.

30. Quick Revision Summary

- Security = Authentication (who) + Authorization (what) + Encryption (safety).
- Spring Security works as a chain of filters before your controller runs.
- Session-based auth = stateful, cookie driven. JWT = stateless, token driven (best for REST/SPA).
- Always hash passwords with BCrypt, never store plain text.
- Use `@PreAuthorize` / URL matchers for RBAC; add ownership checks to prevent IDOR.
- Enable CORS for cross-origin frontend calls; disable CSRF only for stateless APIs.
- 401 = not authenticated, 403 = authenticated but not allowed.
- Use refresh tokens for long sessions, MFA for high-value actions, and rate limiting on login.
- Secure Actuator endpoints and keep secrets out of source code.
- Apply defense in depth — never rely on a single layer of protection.